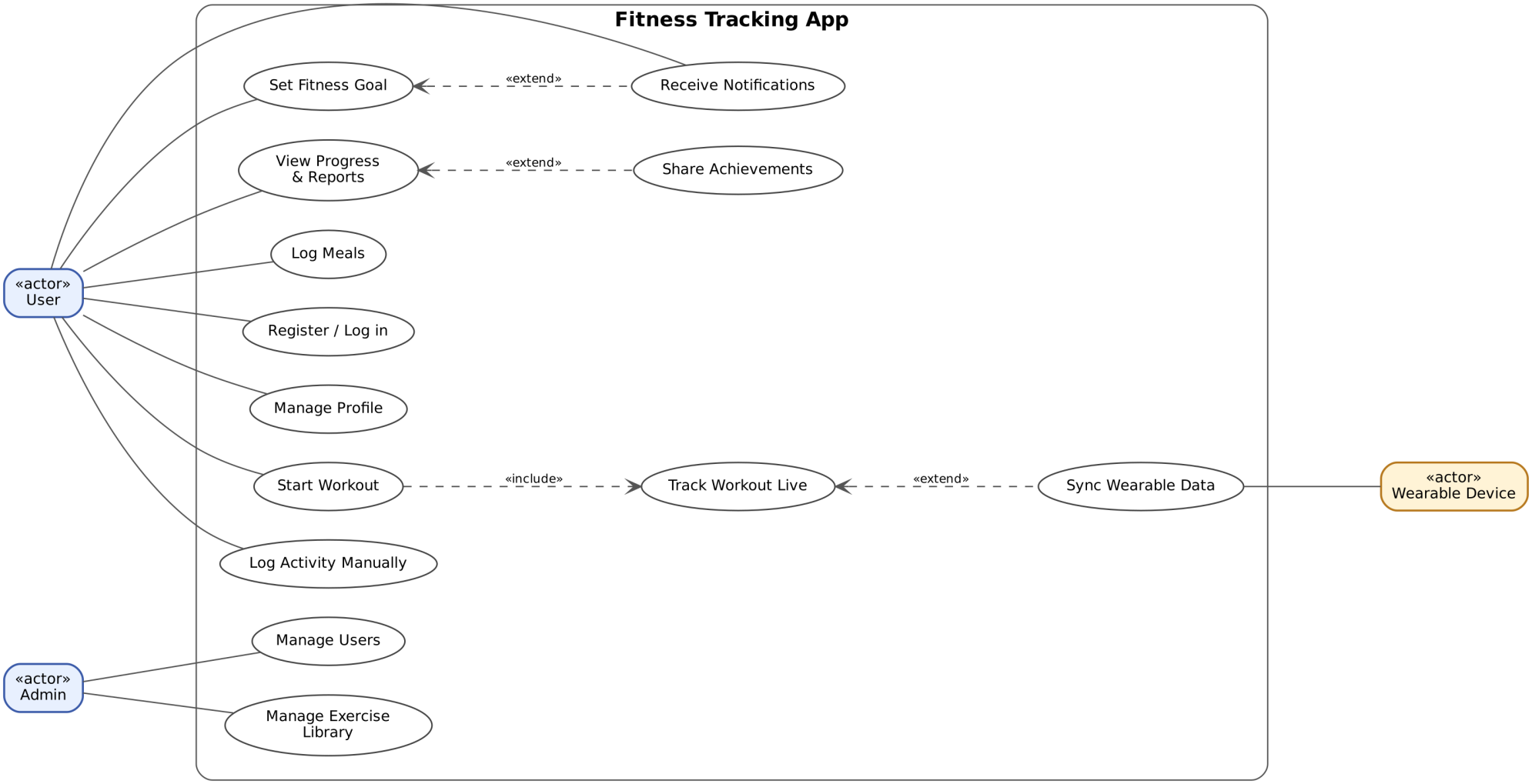
# Fitness Tracking App

UML diagram set - October 2026

| # | Diagram | What it shows |
|---|---|---|
| 1 | **Use Case Diagram** | What the User, Admin and Wearable Device can do with the system. |
| 2 | **Class Diagram** | Core domain classes, attributes, operations and multiplicities. |
| 3 | **Sequence Diagram - Complete a Workout** | Completing a workout, from live sensor data to goal update and notification. |
| 4 | **Activity Diagram - Workout Flow** | End-to-end flow from app launch to the workout summary. |
| 5 | **State Diagram - Workout Session** | Lifecycle of a workout session, including the offline sync path. |
| 6 | **Component Diagram** | Mobile modules, backend services, data stores and external systems. |

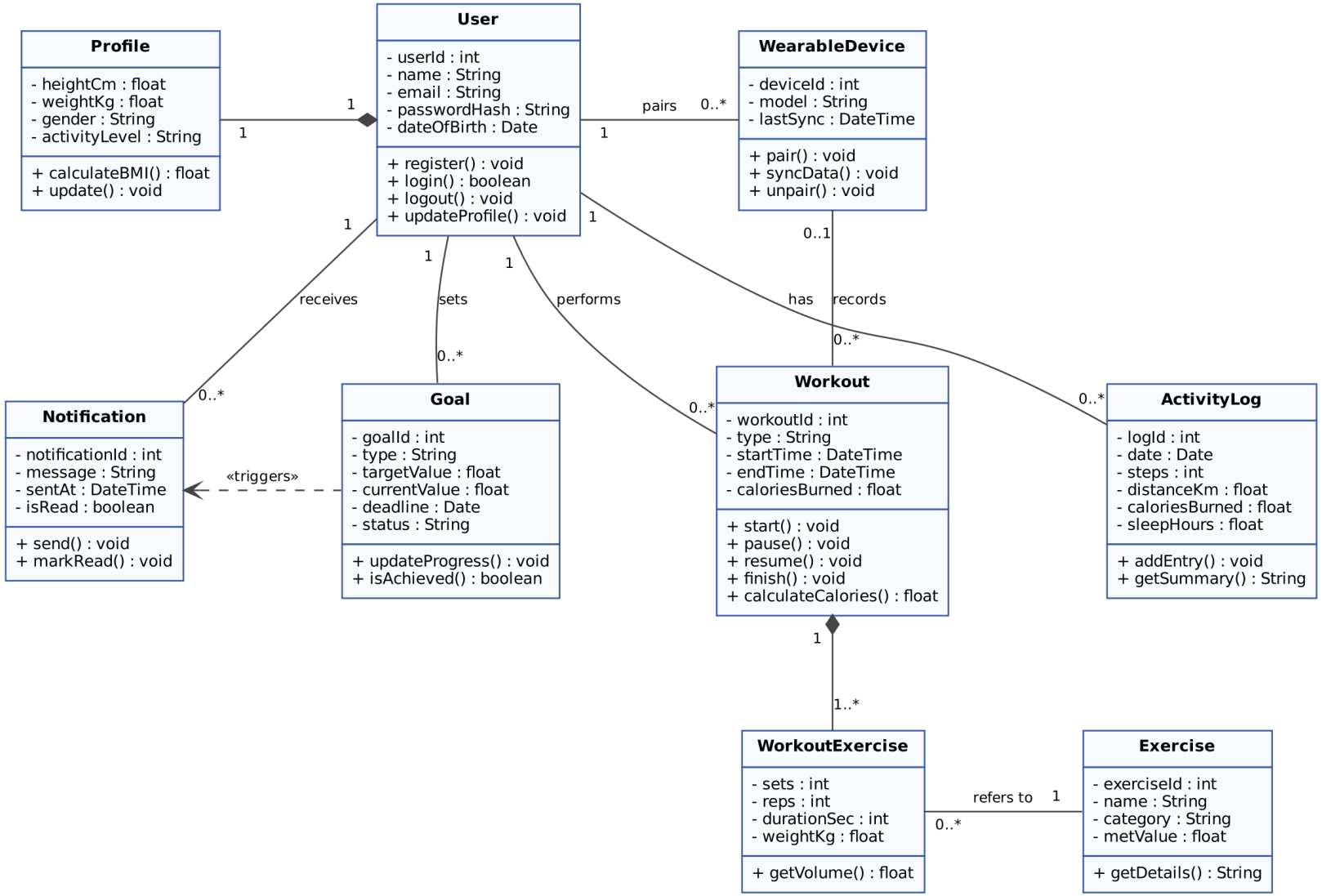
# Use Case Diagram

Three actors interact with the system. Solid lines are associations; dashed arrows show include and extend relationships between use cases.



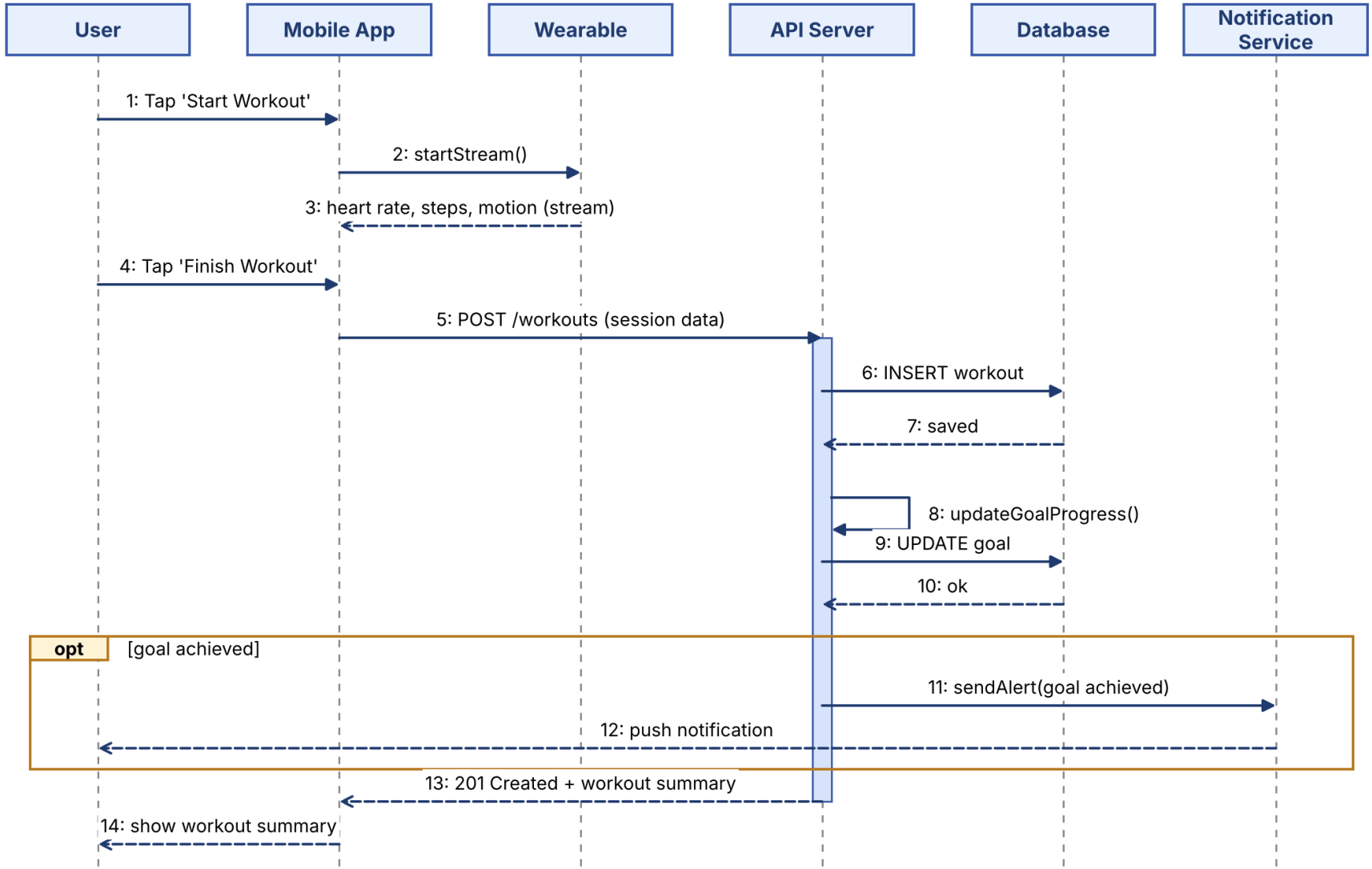
# Class Diagram

A User owns one Profile and many workouts, goals, activity logs, devices and notifications. Each Workout is composed of WorkoutExercise entries that reference the shared Exercise library.



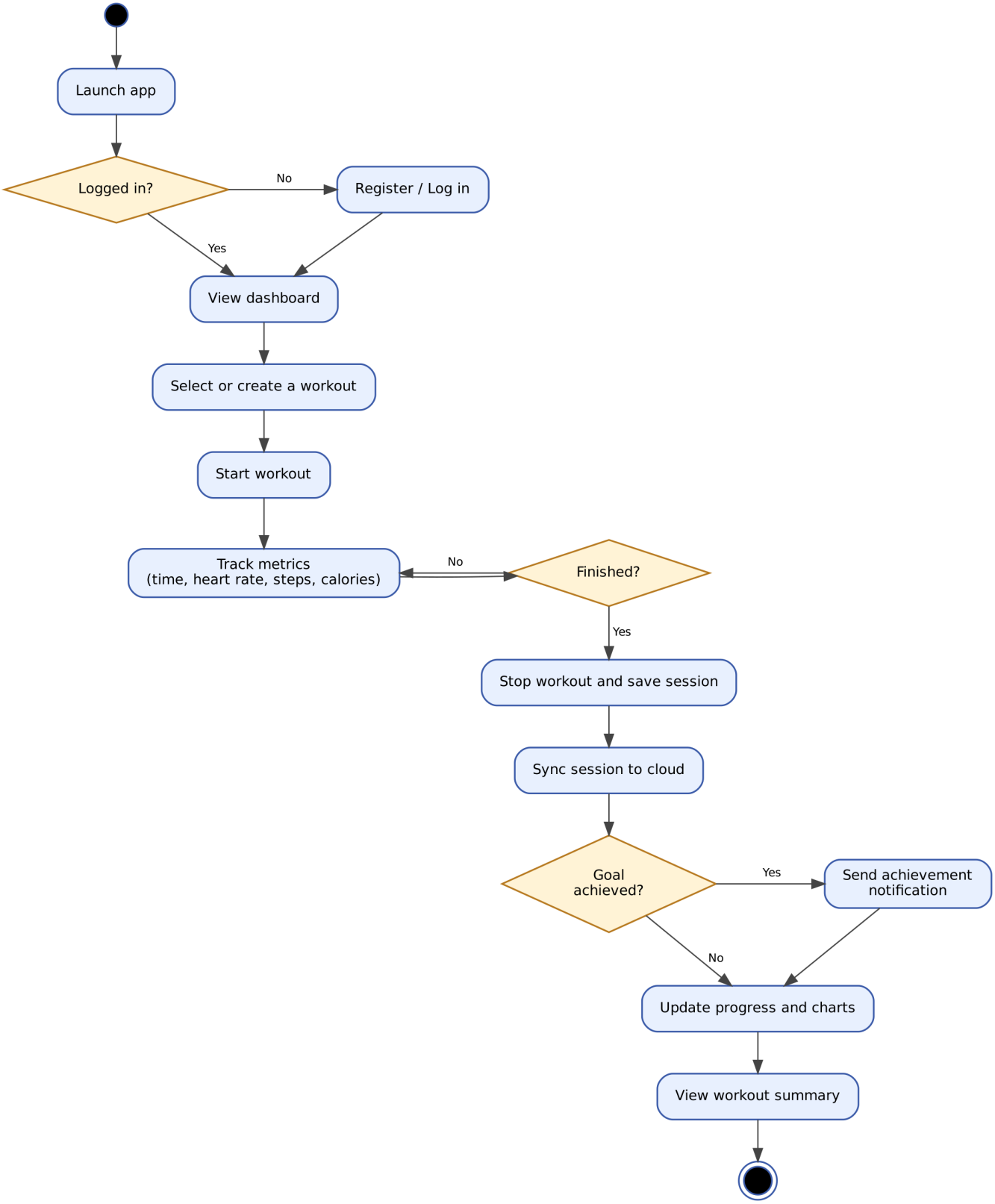
# Sequence Diagram - Complete a Workout

Sensor data streams from the wearable while the workout runs. On finish, the session is saved through the API, goal progress is recalculated, and a notification is sent only if a goal is achieved.



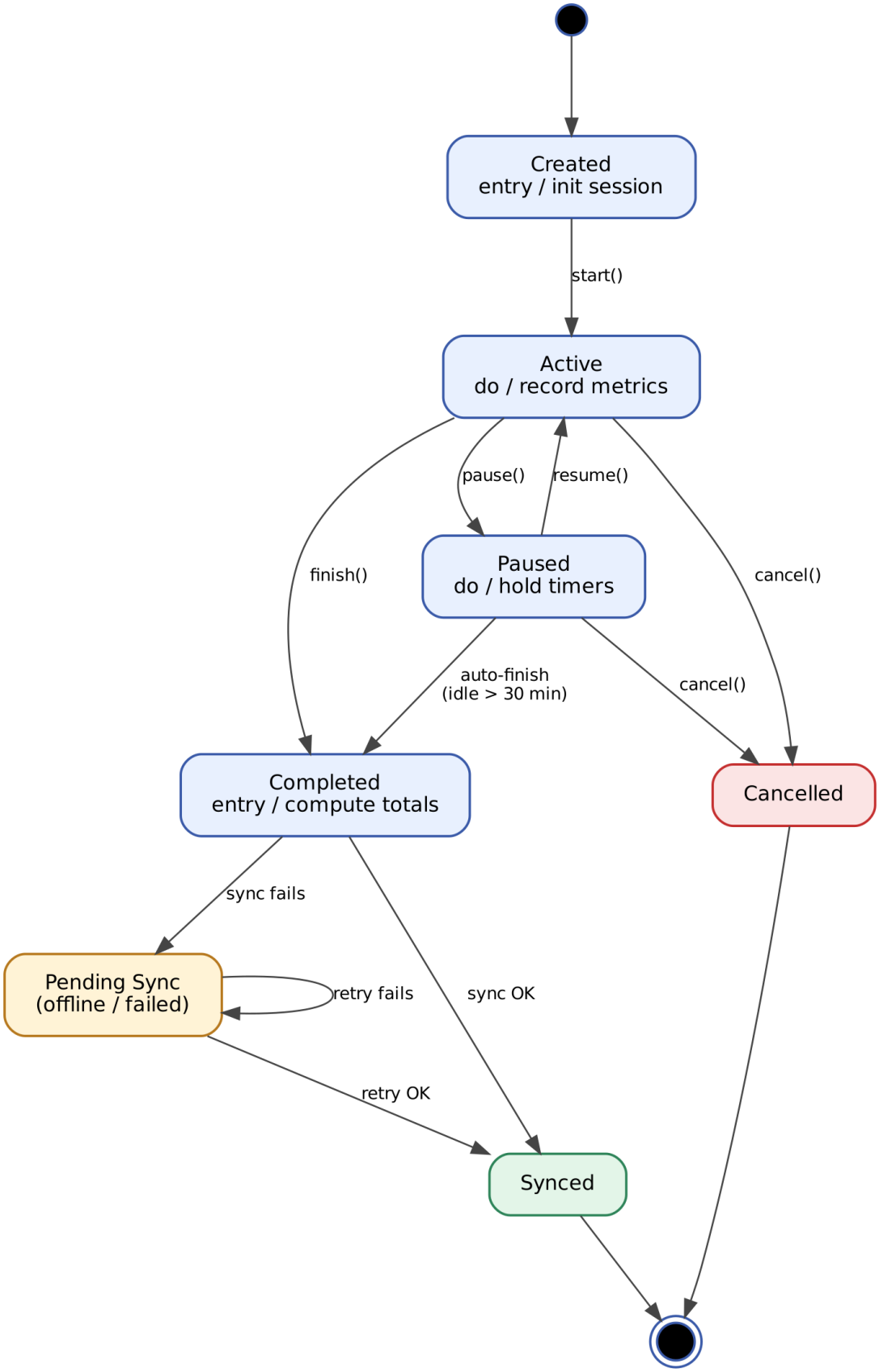
# Activity Diagram - Workout Flow

Covers login, the live tracking loop, cloud sync and the goal-achievement decision.



# State Diagram - Workout Session

A session moves from Created to Active, can be paused and resumed, then ends as Completed or Cancelled. Completed sessions go to Synced, or to Pending Sync and retry when offline.



# Component Diagram

The mobile app collects sensor data and stores it locally, then syncs over HTTPS to the API Gateway. Backend services share a PostgreSQL store, and the Notification Service sends pushes via FCM / APNs.

